Cost-of-Living Data Platform

Cloud Pipeline, Search Taxonomy, and Analytics API

Project Contributors

2026

Abstract

The cloud data platform monitors Australian cost-of-living pressure through public social discussion, media coverage and official CPI indicators. It collects public records from Bluesky, Mastodon and GDELT GKG archive files, stores source-specific raw streams in Elasticsearch, normalises them into a unified raw index, processes text with topic classification and VADER sentiment, and serves analytics through a FastAPI REST API. ABS Monthly CPI observations provide an official comparison signal rather than a causal label. The public architecture presents a single API serving path and uses Fission only for scheduled pipeline jobs, avoiding duplicate API deployments.

Contents

1 Scenario and Motivation

Cost-of-living pressure affects household decisions around rent, groceries, fuel, energy, healthcare, transport, education and debt. Official indicators such as CPI provide a stable statistical view, but they are released on a schedule and do not directly show what people and media outlets are discussing day by day. Public social posts and media records are timely but noisy, inconsistent and hard to inspect manually at scale.

The platform therefore focuses on a repeatable monitoring workflow:

- collect public source records on a schedule;
- filter for Australian context and cost-of-living relevance;
- preserve raw records and processing state;
- process text into topics, sentiment labels and quality flags;
- expose chart-ready analytics through a REST API;
- compare selected discussion signals with official ABS CPI observations.

The aim is not to infer a representative measure of hardship. The aim is to build an inspectable data system that separates social discussion, media coverage and official indicators.

2 Data Sources and Search Terms

Table 1: Data sources

Source	Role	Interpretation
Bluesky	Public social discussion	Timely public posts, filtered for Australian context. Not population-representative.
Mastodon	Public social discussion	Public posts from multiple instances, useful as a second social signal.
GDELT GKG	Media coverage	Public 15-minute archive files used as media attention. It is not treated as public sentiment.
ABS Monthly CPI	Official indicator	Monthly official price data for comparison, not causal modelling.

The collection taxonomy is explicit and versioned in `data/cost_of_living_keywords.csv`. It includes housing, groceries, energy, fuel, transport, eating out, healthcare, home goods, education, inflation, wages and debt. Terms were selected to cover common Australian cost categories, official CPI-aligned categories and locally meaningful signals such as major supermarket names, public transport terms and healthcare references. Source-specific collectors follow the public Bluesky, Mastodon and GDELT interfaces [2, 7, 8], while official monthly indicators are obtained from ABS CPI documentation [1].

For social sources, a record must have a cost-of-living signal and Australian context. For GDELT, filtering is stricter because GKG records can contain metadata-heavy snippets. Incremental harvesting and historical backfill both use the same GKG archive processor: it reads the public master file list, downloads `.gkg.csv.zip` archives, verifies md5 checksums, extracts CSV rows, filters for relevant Australian cost-of-living records and bulk indexes matching rows. GDELT is analysed as media coverage, while Bluesky and Mastodon are grouped as social discussion.

3 Architecture

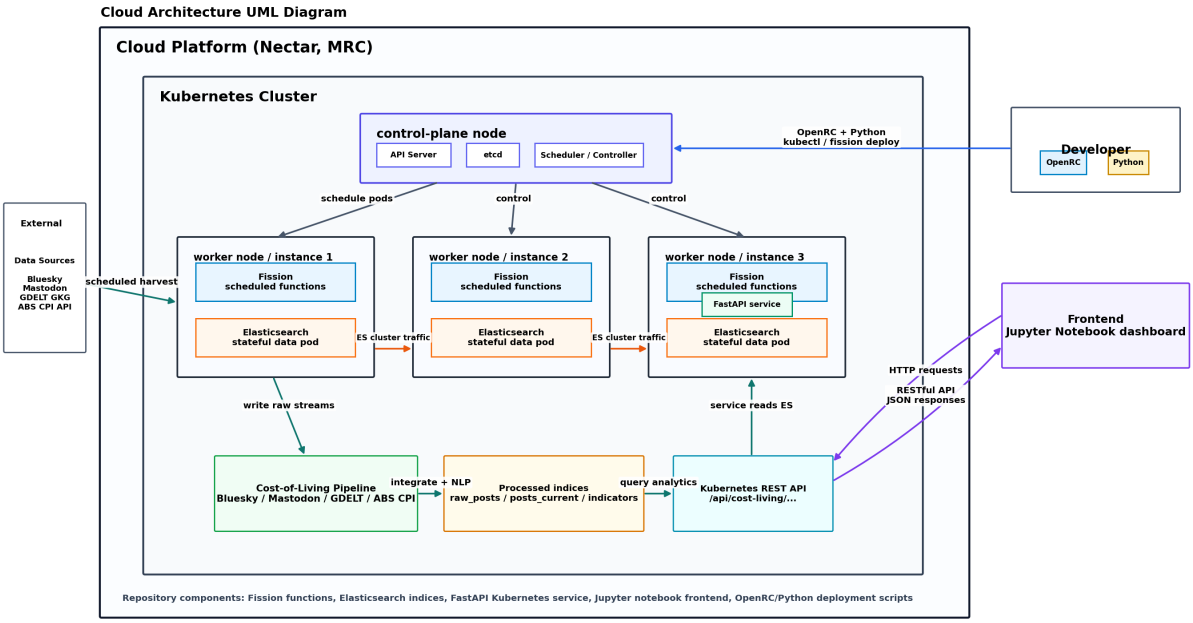

Figure 1: System architecture

The public architecture has one API layer. FastAPI serves all analytics endpoints. Fission is used for scheduled ingestion, raw integration and CPI jobs only. NLP processing runs as Kubernetes workers scaled by KEDA from a Redis queue. This avoids the operational ambiguity of deploying the same analytics API as both a monolithic service and separate HTTP functions. Kubernetes, Fission and Elasticsearch provide orchestration, scheduled function execution and search storage respectively [6, 4, 3].

Table 2: Runtime responsibilities

Component	Responsibility
Fission harvesters	Collect Bluesky, Mastodon, GDELT and ABS CPI data on schedules.
Raw integrator	Normalise source-specific records into the unified raw schema and enqueue NLP work.
KEDA NLP worker	Scale from the Redis queue, claim pending records, clean text, classify topics and run sentiment.
Elasticsearch	Store raw streams, raw processing state, ILM-managed processed documents, aliases and official indicators.
FastAPI	Serve REST endpoints, Prometheus metrics, rate limits and request-id traced logs.
Jupyter notebooks	Consume the API and render exploratory charts.

3.1 Elasticsearch Data Model

Table 3: Core indices and aliases

Index or alias	Purpose
`cost_living_bluesky_raw_stream`	Bluesky source-specific raw records.
`cost_living_mastodon_raw_stream`	Mastodon source-specific raw records.
`cost_living_gdelt_raw_stream`	GDELT source-specific raw records.
`cost_living_raw_posts`	Unified raw records and NLP processing state.
`cost_living_processed_posts_write`	Processed write alias managed by ILM rollover.
`cost_living_processed_posts-*`	Processed topic and sentiment backing indices.
`cost_living_posts_current`	Stable API read alias.
`cost_living_indicators`	ABS CPI observations.
`cost_living_monthly_topic_metrics`	Optional monthly rollup.

The raw integrator pushes lightweight NLP work items into Redis after it writes new unified raw records. KEDA watches this Redis list and scales the NLP worker deployment from zero. Worker queue messages have a bounded retry budget, and exhausted or malformed messages are isolated in a Redis dead-letter queue for inspection. The worker still uses Elasticsearch status fields for safe claiming: raw records start as `pending`, a worker atomically moves a bounded set to `processing`, then writes processed records and updates raw status to `processed`, `discarded` or `error`. Stale processing records can be retried. Sentiment scoring uses VADER because it is lightweight and interpretable for short social text [5].

4 API Design

The API is read-only and exposes analytics resources under:

`/api/cost-living`

The application also accepts internal `/api/...` paths for local tests. Common filters include source group, platform, topic, date range, period, quality and quality-flag exclusion.

Table 4: Endpoint groups

Group	Examples
Health and pipeline	`/health`, `/pipeline/status`, `/pipeline/queues`
Overview and trends	`/stats/overview`, `/trends/documents`, `/trends/categories`
Topic analytics	`/categories/counts`, `/categories/sentiment`, `/categories/share`
Data quality	`/data-quality/summary`, `/data-quality/comparison`
Media and platform views	`/media/coverage`, `/platforms/categories`
Official comparison	`/official/comparison`
Diagnostics	`/logs/errors`, `/metrics`, `/rate-limit/status`

This style is analytics-resource oriented rather than RPC-oriented command naming. The API uses `GET` because clients read aggregates and do not mutate platform state.

5 Implementation Quality

The codebase uses Pydantic settings, typed Python, explicit Elasticsearch mappings, a shared source registry and focused pytest coverage. The public version also removes duplicate Fission HTTP API functions so the deployment story is consistent.

Important reliability choices include:

- stable ids for idempotent writes;
- separate raw and processed indices;
- explicit processing state fields;
- ILM-managed processed backing indices with separate write and read aliases;
- Redis-backed short API response caching with an in-memory fallback to reduce repeated aggregation load;
- Prometheus metrics for API request count, latency, cache and rate-limit behaviour;
- request-id propagation and structured request logging in the FastAPI layer;
- Redis-backed job locks, lifecycle events, run identifiers and queue-backed KEDA worker scaling;
- Kubernetes ConfigMaps and Secrets for environment-specific configuration.

6 Validation and Results

The repository test suite covers harvester helpers, ABS CPI parsing, GDELT archive processing, GDELT backfill resume behaviour, source registry logic, NLP processing, Redis runtime queue behaviour, API cache behaviour, rate limiting, queue workers, Elasticsearch lifecycle templates, analytics store behaviour and API route wiring. The current public branch passes:

```
57 pytest tests passing
```

The original cloud run processed approximately 245k raw documents, 225k processed documents and 10,134 ABS CPI observations. These counts are a snapshot of the historical run, not a live service guarantee.

Figure 2: Raw document processing outcomes

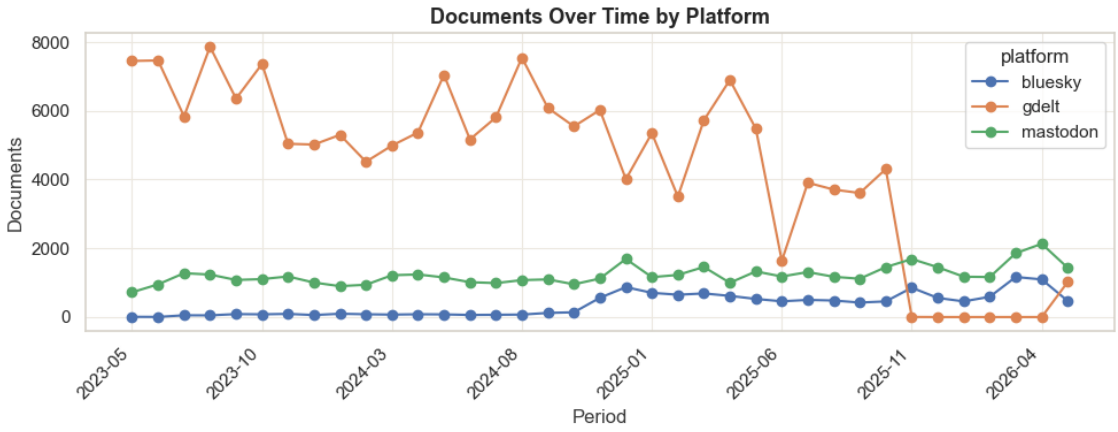

Figure 3: Documents over time by platform

6.1 Topic and Sentiment Patterns

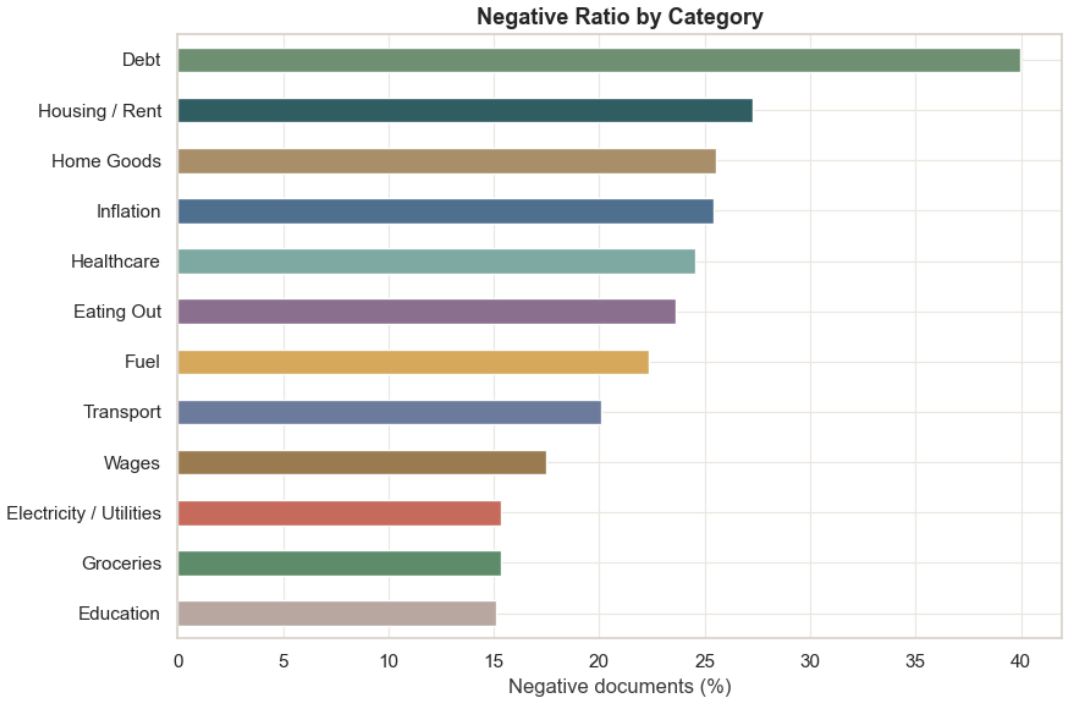

Figure 4: Negative ratio by category

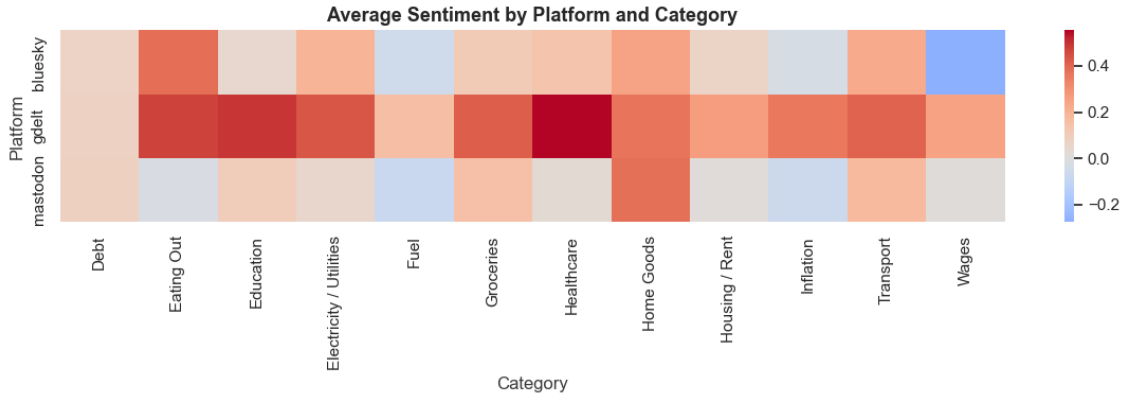

Figure 5: Platform and category sentiment heatmap

The charts show that cost-of-living discussion is not uniform across categories or platforms. Social sources and GDELT should be interpreted separately: social sources are closer to personal expression, while GDELT measures media coverage.

6.2 Official CPI Comparison

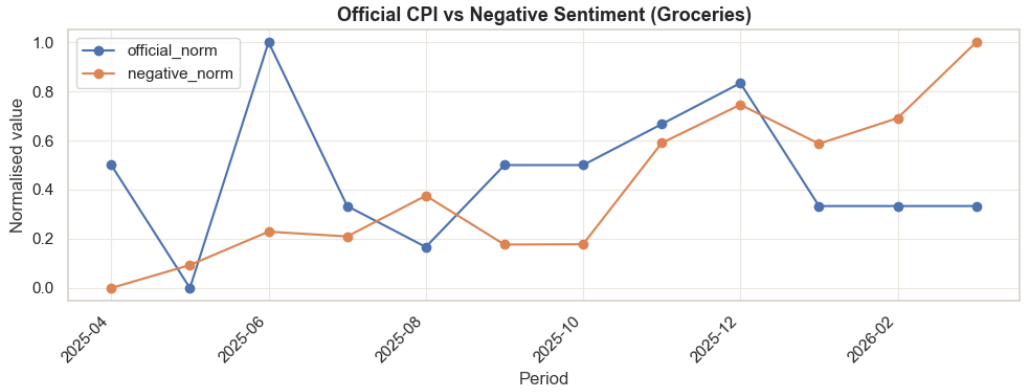

Figure 6: Groceries: official CPI and negative sentiment comparison

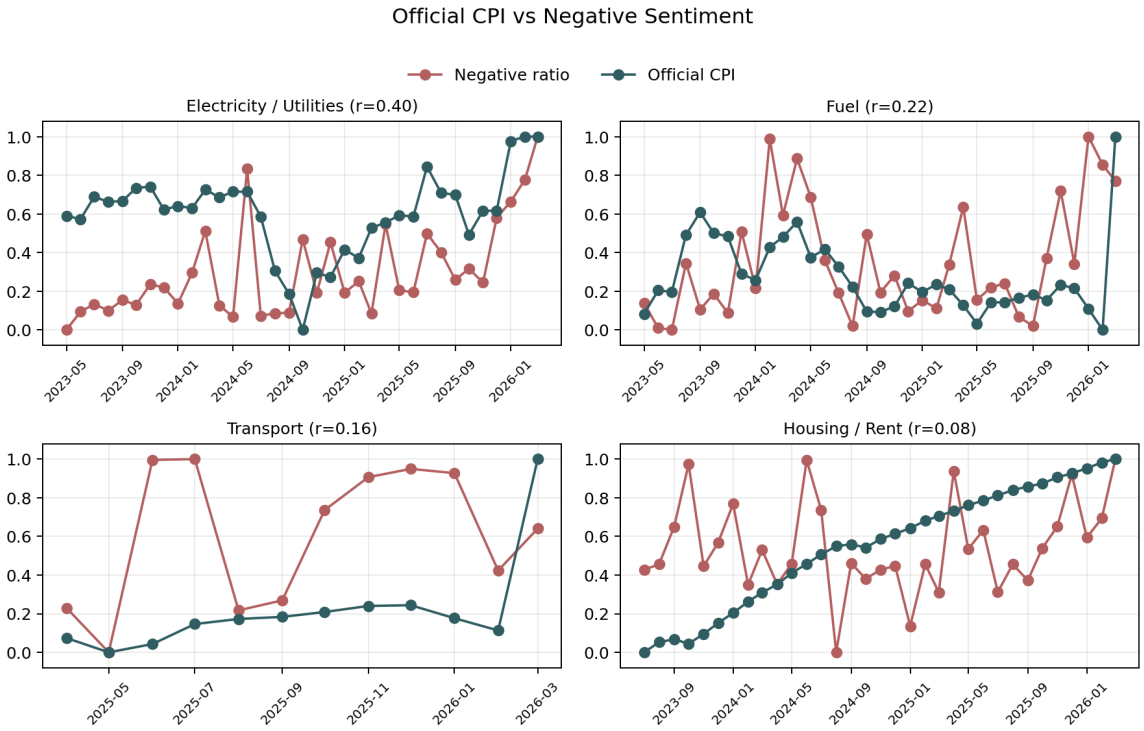

Figure 7: Official CPI and sentiment comparison across selected topics

The CPI comparison is descriptive. CPI measures official price movement, while social and media records measure attention and language. Alignment between the two is useful evidence for interpretation but does not imply causation.

7 Limitations

- Public social platforms are not representative population samples.
- GDELT records can be noisy and metadata-heavy.
- Keyword collection can miss relevant posts that use unexpected wording.
- Topic classification assigns a primary category and can understate cross-topic discussion.
- VADER is transparent and lightweight, but it can misread sarcasm and political language.
- ABS CPI is monthly and delayed relative to online discussion.
- The dataset does not contain reliable city-level fields.

8 Conclusion

The platform demonstrates a maintainable cloud analytics pattern for public text and official indicator data. The strongest engineering choices are the separation of raw and processed data, explicit source registration, archive-based GDELT ingestion for both incremental and historical runs, consistent index configuration, a single API serving path, Redis-backed runtime diagnostics and API caching, queue-backed KEDA NLP scaling, ILM-managed processed indices, and a testable FastAPI interface. The main analytical caution is that each source measures a different phenomenon: social discussion, media coverage and official price indicators should be compared carefully rather than merged into one sentiment claim.

References

[1] Australian Bureau of Statistics. ABS Data API and Consumer Price Index. https://www.abs.gov.au/, 2026. Official CPI data source documentation.

[2] Bluesky. AT Protocol and Bluesky API Documentation. https://docs.bsky.app/, 2026. Public post collection interface documentation.

[3] Elastic. Elasticsearch Documentation. https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html, 2026. Mappings, aliases, aggregations and date histogram documentation.

[4] Fission. Fission Documentation. https://fission.io/docs/, 2026. Function package and timer trigger documentation.

[5] C. J. Hutto and Eric Gilbert. VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. In *Proceedings of the International AAAI Conference on Web and Social Media*, 2014.

[6] Kubernetes. Kubernetes Documentation. https://kubernetes.io/docs/, 2026. Deployment, service, ConfigMap and Secret documentation.

[7] Mastodon. Mastodon API Documentation. https://docs.joinmastodon.org/api/, 2026. Public API documentation.

[8] The GDELT Project. GDELT 2.0 and GKG Documentation. https://www.gdeltproject.org/, 2026. Media data source documentation.